

Evaluación Práctica

Sistema de Gestión de Consolas de Videojuegos

Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
FPY1101	FUNDAMENTOS DE PROGRAMACION	7 horas	25%

1. Instrucciones

Descripción

- Esta evaluación tiene como objetivo medir los resultados de aprendizaje de la Experiencia de Aprendizaje 3.
- La evaluación contempla 1 ejercicio que medirá distintos contenidos.
- El tiempo asignado para desarrollar esta evaluación en laboratorio es de 7 horas pedagógicas.
- La evaluación se realiza de manera individual.
- Las instrucciones se proporcionarán presencialmente el día de la evaluación.
- **Esta evaluación consiste en una ejecución práctica que medirá los siguientes Indicadores de Logro:**
 - IL 3.1 – Identifica arreglos que permitan el almacenamiento de datos según los requerimientos del problema planteado.
 - IL 3.2 – Utiliza arreglos para la inserción, eliminación, modificación y búsqueda de datos temporales.
 - IL 4.1 – Programa funciones que permitan la reutilización de código según el problema planteado.
 - IL 4.2 – Integra funciones invocadas desde el programa principal para dar solución al problema planteado.
- La evaluación se centra en el desarrollo de un programa en Python para dar respuesta al problema planteado.

2. Enunciado del Problema

Una cadena de tiendas de videojuegos necesita un sistema para administrar su catálogo de consolas. El sistema debe manejar dos estructuras de datos relacionadas:

Estructura de Datos Requerida

Diccionario consolas → relaciona la sigla de cada consola con sus datos descriptivos:

- Clave: sigla de la consola (str) — ejemplo: "PS5", "XSX", "NSW"
- Valor: lista con [nombre (str), fabricante (str), año_lanzamiento (int)]

Diccionario ventas → relaciona la misma sigla con información comercial:

- Clave: sigla de la consola (str) — debe coincidir con las claves de consolas
- Valor: lista con [precio (float), stock (int)]

Ejemplo de estructura inicial (solo referencia, NO copiar en el código):

```
consolas = {  
    "PS5": ["PlayStation 5", "Sony", 2020],  
    "XSX": ["Xbox Series X", "Microsoft", 2020],  
    "NSW": ["Nintendo Switch", "Nintendo", 2017]  
}  
  
ventas = {  
    "PS5": [649990.0, 15],  
    "XSX": [599990.0, 8],  
    "NSW": [299990.0, 30]  
}
```

2.1 Requerimientos del Sistema

El programa debe implementar un menú interactivo con las siguientes opciones:

Menú Principal

1. Agregar consola
2. Buscar consola por sigla
3. Eliminar consola
4. Mostrar todas las consolas
5. Salir

2.2 Descripción Detallada de Funcionalidades

Opción 1 – Agregar consola

Solicitar al usuario los siguientes datos y validar cada uno antes de agregar:

- Sigla: entre 2 y 5 caracteres, solo letras mayúsculas, no puede existir previamente.
- Nombre: entre 3 y 40 caracteres, no puede estar vacío.
- Fabricante: entre 2 y 30 caracteres, no puede estar vacío.
- Año de lanzamiento: número entero entre 1972 y 2025.
- Precio: número decimal mayor a 0.
- Stock: número entero mayor o igual a 0.

Si todos los datos son válidos, se debe agregar la consola en ambos diccionarios bajo la misma clave (sigla). Si la sigla ya existe, mostrar mensaje de error y no agregar.

Opción 2 – Buscar consola por sigla

Solicitar la sigla al usuario y buscarla en el diccionario consolas. Si existe, mostrar todos los datos combinados de ambos diccionarios con el siguiente formato:

```
=== Consola Encontrada ===
Sigla      : PS5
Nombre     : PlayStation 5
Fabricante : Sony
Año lanz.  : 2020
Precio     : $649,990.00
Stock      : 15 unidades
```

Si la sigla no existe, mostrar un mensaje indicando que no se encontró.

Opción 3 – Eliminar consola

Solicitar la sigla al usuario. Invocar la función de búsqueda para verificar si existe. Si la consola es encontrada, eliminar la entrada en ambos diccionarios simultáneamente y confirmar con un mensaje. Si no existe, informar al usuario.

Opción 4 – Mostrar todas las consolas

Recorrer ambos diccionarios de forma sincronizada mediante la clave compartida (sigla) y mostrar la información completa de cada consola. Si no hay consolas registradas, mostrar un mensaje indicando que el sistema está vacío.

Formato de salida esperado por consola:

```
=====
LISTADO COMPLETO DE CONSOLAS
=====
Sigla: PS5 | PlayStation 5 | Sony | 2020 | $649,990.00 | Stock: 15
Sigla: XSX | Xbox Series X | Microsoft | 2020 | $599,990.00 | Stock: 8
Sigla: NSW | Nintendo Switch | Nintendo | 2017 | $299,990.00 | Stock: 30
=====
Total de consolas: 3
```

2.3 Restricciones Técnicas Obligatorias

El programa DEBE cumplir con las siguientes restricciones

- Ambos diccionarios deben inicializarse vacíos al inicio del programa (sin datos de prueba).
- Cada funcionalidad debe implementarse en funciones separadas con responsabilidad única.
- Las funciones de validación deben recibir el dato a validar y retornar True o False únicamente, sin imprimir mensajes.
- Los mensajes de error ante datos inválidos deben mostrarse en la función que solicita los datos, no en las funciones de validación.
- Los diccionarios deben pasarse como argumentos a las funciones; NO se permite el uso de variables globales.
- La función de eliminar debe invocar la función de búsqueda, sin duplicar su lógica.
- El menú debe mostrarse en cada iteración del ciclo principal, que permanece activo hasta que el usuario seleccione la opción de salida.

3. Entrega mediante GitHub

El proyecto debe gestionarse con control de versiones usando Git y publicarse en GitHub. A continuación se detallan los requisitos de entrega.

3.1 Requisitos del Repositorio

Configuración obligatoria del repositorio

- Crear un repositorio público en GitHub con el nombre:
 - **FPY1101_EP_Consolas_<NombreApellido>** (ejemplo: FPY1101_EP_Consolas_JuanPerez)
- El repositorio debe contener obligatoriamente los siguientes archivos:
 - `consolas.py` → archivo principal con todo el código fuente
 - `README.md` → descripción del proyecto (ver punto 3.2)
- El archivo `.py` debe ejecutarse sin errores desde la terminal.
- No se aceptarán repositorios privados ni entregas por correo o USB.

3.2 Contenido del README.md

El archivo `README.md` debe redactarse en español e incluir las siguientes secciones:

```
# Sistema de Gestión de Consolas de Videojuegos

## Descripción
Breve descripción del sistema y su propósito.

## Funcionalidades
- Agregar consola
- Buscar consola por sigla
- Eliminar consola
- Mostrar todas las consolas

## Cómo ejecutar
Instrucciones para correr el programa desde la terminal.

## Autor
Nombre completo del estudiante.
```

3.3 Flujo de Trabajo con Git

Se evaluará que el historial de commits refleje el desarrollo progresivo del programa. Requisitos mínimos:

Commits mínimos requeridos (en orden cronológico)

6. Commit inicial: estructura de diccionarios y menú principal
7. Commit: implementación de funciones de validación
8. Commit: función agregar consola

9. Commit: función buscar y eliminar consola
10. Commit: función mostrar todas y ciclo principal
11. Commit final: README.md y revisión general

Importante: un único commit con todo el código equivale a Desempeño No Logrado en el criterio de control de versiones.

3.4 Formato de Entrega del Link

Una vez finalizado el desarrollo, el estudiante debe entregar el link al repositorio a través de la plataforma indicada por el docente. El link debe tener el siguiente formato:

```
https://github.com/<usuario>/<nombre-repositorio>
```

Ejemplo:

```
https://github.com/juanperez/FPY1101_EP_Consolas_JuanPerez
```

Checklist antes de enviar el link

Verificar cada punto antes de enviar:

- ☐ El repositorio es público y se puede acceder sin iniciar sesión.
- ☐ El archivo consolas.py está en la raíz del repositorio.
- ☐ El programa corre sin errores con `python consolas.py`.
- ☐ El README.md contiene todas las secciones requeridas.
- ☐ El historial tiene al menos 6 commits con mensajes descriptivos.
- ☐ El link fue copiado correctamente (sin espacios ni caracteres extra).

4. Pauta de Evaluación: Lista de Cotejo

Categoría	% Logro	Descripción
Desempeño Logrado	100%	Cumple completamente con la ejecución del estándar.
Desempeño No Logrado	0%	No alcanza el estándar; presenta dificultades o errores en la ejecución.

N°	Indicador de Evaluación	Logrado	No Logrado	Pond.
1	Define los dos diccionarios (consolas y ventas) con las claves correctas y los tipos de datos especificados en el enunciado antes de comenzar a operar.			3%
2	Inicializa las listas de valores de cada clave del diccionario consolas con los tipos de dato correctos (str, str, int) y del diccionario ventas (float, int).			3%
3	Programa la lógica de agregar una consola validando que la clave (sigla) no exista previamente en ninguno de los dos diccionarios.			5%
4	Almacena correctamente la entrada en ambos diccionarios (consolas y ventas) de forma sincronizada bajo la misma clave.			3%
5	Programa una lógica de búsqueda que retorne True/False o la clave encontrada según la condición definida en el enunciado.			4%
6	Programa el criterio de eliminación utilizando el resultado de búsqueda y elimina la entrada en ambos diccionarios simultáneamente.			4%
7	Programa la lógica de recorrido cruzado que muestra los datos de ambos diccionarios combinados por clave.			5%
8	Usa técnica correcta de recorrido de diccionarios (items(), keys() o índice directo según corresponda).			4%
9	Define una función para mostrar el menú de opciones en pantalla acorde a las opciones planteadas en el enunciado.			3%
10	Define una función para leer y validar la opción del menú ingresada por el usuario.			3%
11	Define una función para agregar consola que recibe ambos diccionarios como parámetros, solicita campos al usuario, invoca funciones de validación y agrega solo cuando todo es válido.			4%
12	Define una función para buscar una consola por sigla acorde a la condición planteada.			4%
13	Define una función para eliminar que invoca la función de búsqueda sin duplicar su lógica, y elimina en ambos diccionarios.			4%
14	Define una función para mostrar todos los registros recorriendo ambos diccionarios de forma cruzada.			3%

15	Define a lo menos una función de validación con responsabilidad única que recibe el dato a validar y retorna True o False.			3%
16	Las condiciones aplicadas en cada función de validación corresponden exactamente a las restricciones del enunciado.			5%
17	Los mensajes de error ante datos inválidos se muestran en la función que solicita los datos, no dentro de las funciones de validación.			4%
18	Implementa la función de mostrar menú y leer opción en cada iteración del ciclo, usando el valor retornado para controlar el flujo.			5%
19	Desarrolla la lógica del programa principal para dirigir la ejecución a la función correspondiente según la opción del menú, incluyendo la opción de salida.			6%
20	Ambos diccionarios se pasan como argumento a todas las funciones que los necesitan, sin acceder a ellos como variables globales.			7%
21	El ciclo principal permanece activo permitiendo múltiples operaciones hasta que el usuario selecciona la opción de salida.			6%
22	Las variables y claves de los diccionarios tienen nombres descriptivos y el código presenta indentación consistente.			3%
23	Crea un repositorio público en GitHub con el nombre correcto y sube el archivo consolas.py en la raíz del repositorio.			2%
24	El repositorio contiene un archivo README.md con las secciones requeridas: descripción, funcionalidades, cómo ejecutar y autor.			2%
25	El historial de commits refleja desarrollo progresivo con al menos 6 commits descriptivos y en orden lógico.			3%
26	Entrega el link del repositorio en el formato correcto y el repositorio es accesible públicamente sin errores.			2%
TOTAL				100%